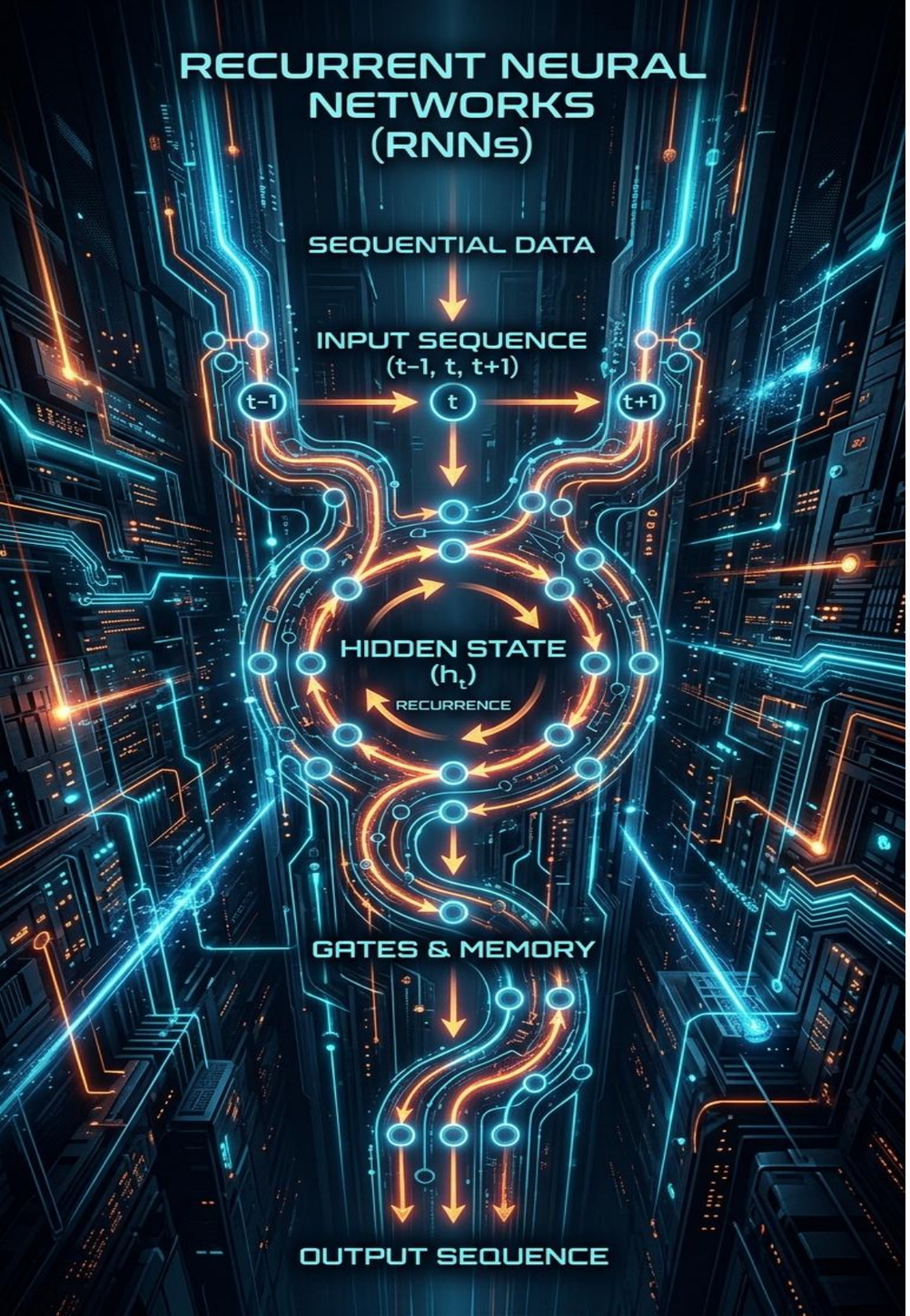


Recurrent Neural Networks (RNNs)

A Complete Lecture on Recurrent Neural Networks

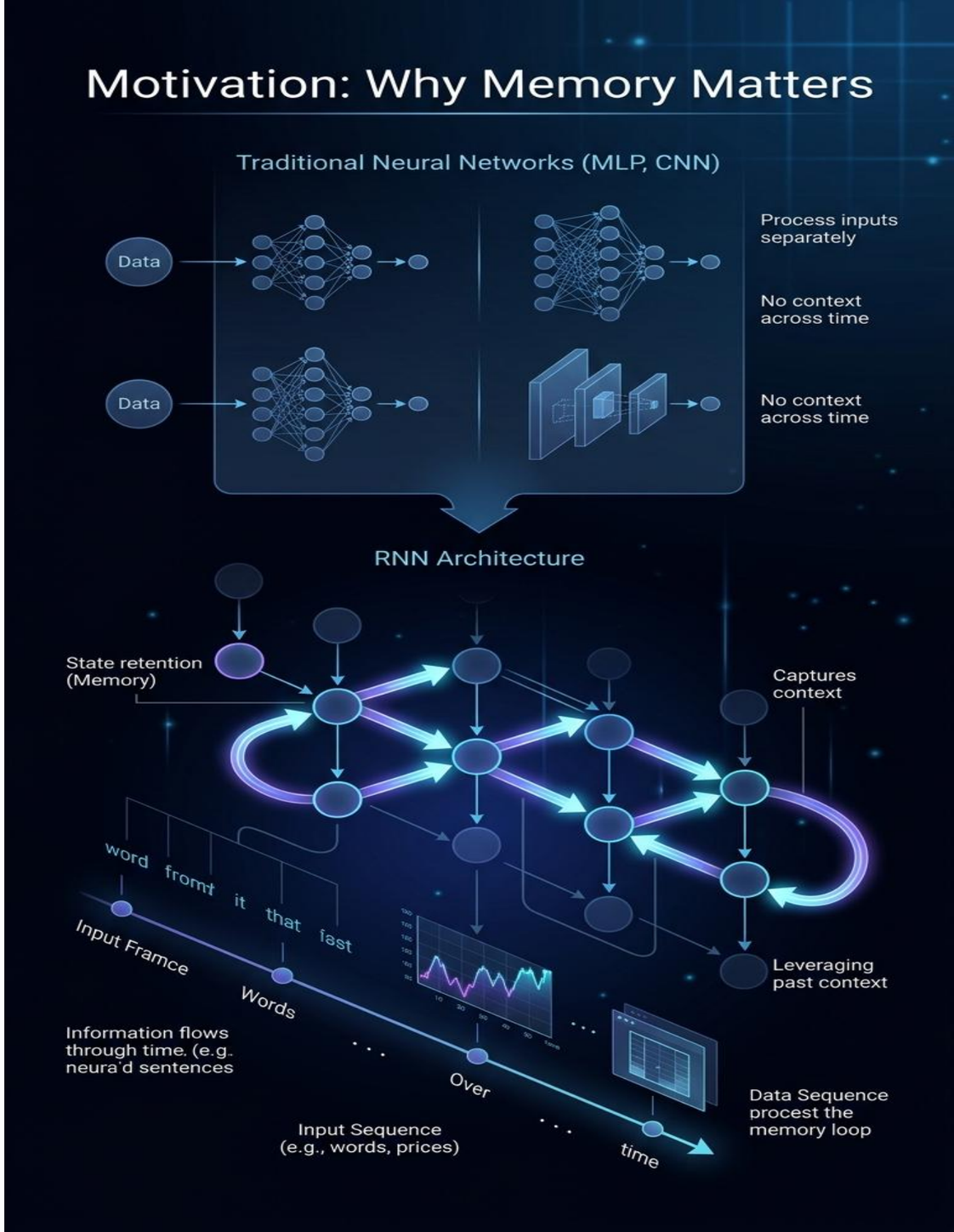
Undergraduate Engineering — Neural Networks & Deep Learning

CHAPTER 0

Prerequisite

Why Memory Matters

- Traditional networks like MLPs and CNNs process inputs independently.
- However, many problems depend on *context* — the relationship between what happened before and what happens next.
- For example, understanding the meaning of a sentence or predicting future stock prices requires remembering previous inputs.
- RNNs address this need by introducing memory through recurrent connections.

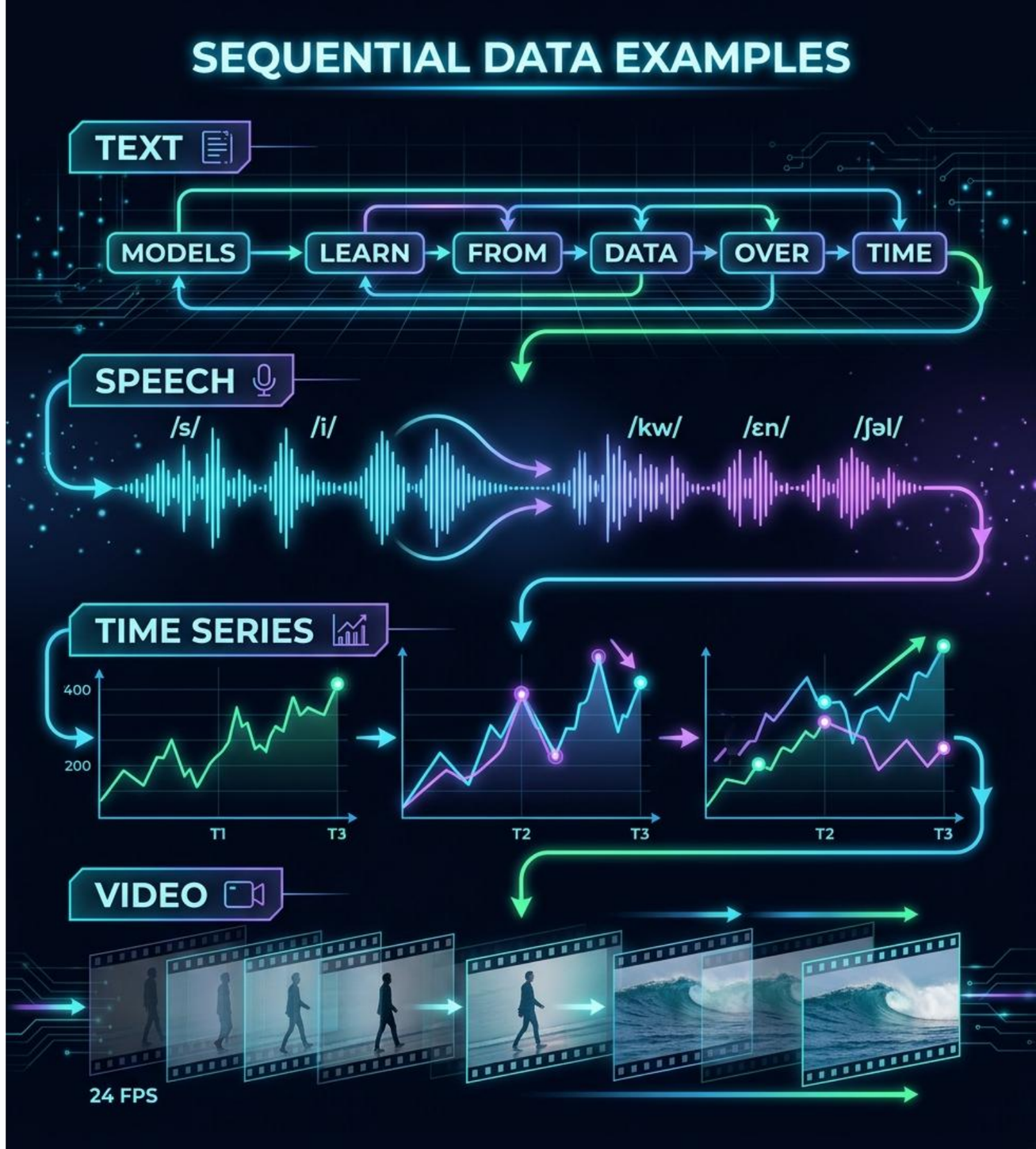


Why Do We Need Memory in Neural Networks?

Many real-world tasks involve inputs that are not isolated but occur as sequences. For example, the meaning of a word in a sentence depends on previous words, just as predicting tomorrow's temperature depends on previous days. Traditional neural networks treat each input independently, meaning they cannot capture relationships that unfold over time. Recurrent Neural Networks fill this gap by enabling the network to store information from earlier steps and use it later.

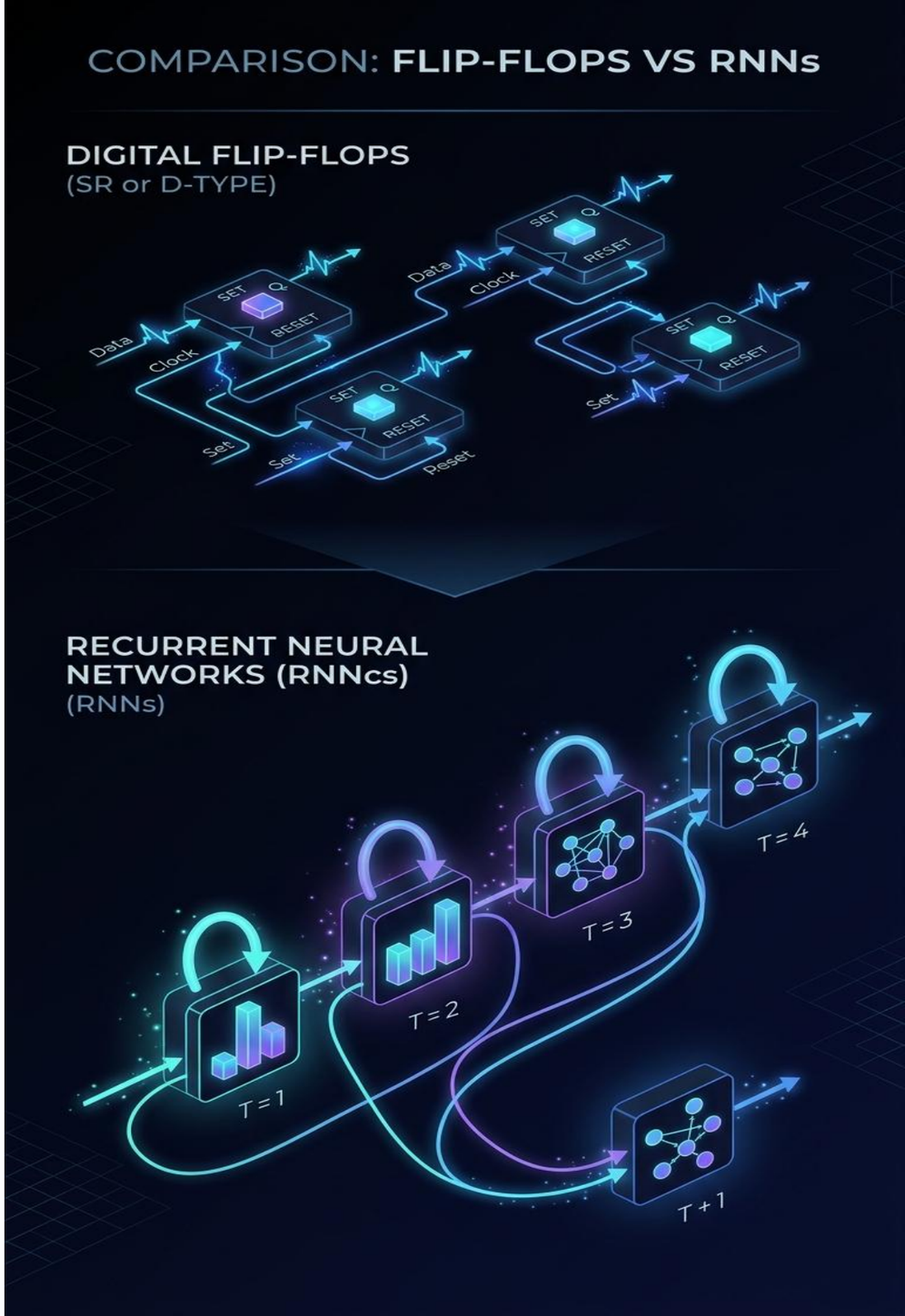
Examples:

Sequential or temporal data includes any information where order matters. Natural language requires understanding the sequence of words. Speech is made up of time-dependent sound waves. Time series data—such as stock prices, ECG signals, or weather patterns—changes step by step. Even video processing relies on understanding how frames evolve over time. These tasks require models that can interpret patterns that span multiple time steps.



Why Do We Need Memory in Neural Networks?

- In digital electronics, flip-flops store binary state variables and use them to influence future outputs. Their behavior depends on the previous state and the current input. This idea is analogous to RNNs: instead of storing a single bit, RNNs store a vector representing past information. Just like flip-flops enable memory in circuits, recurrent connections enable memory in neural networks.
- A Recurrent Neural Network is a neural architecture where outputs from previous steps are fed back into the network at the next step. This feedback loop forms a “memory” that allows the network to use information from earlier inputs. RNNs process sequences one element at a time while updating a hidden state that summarizes what has been observed so far.

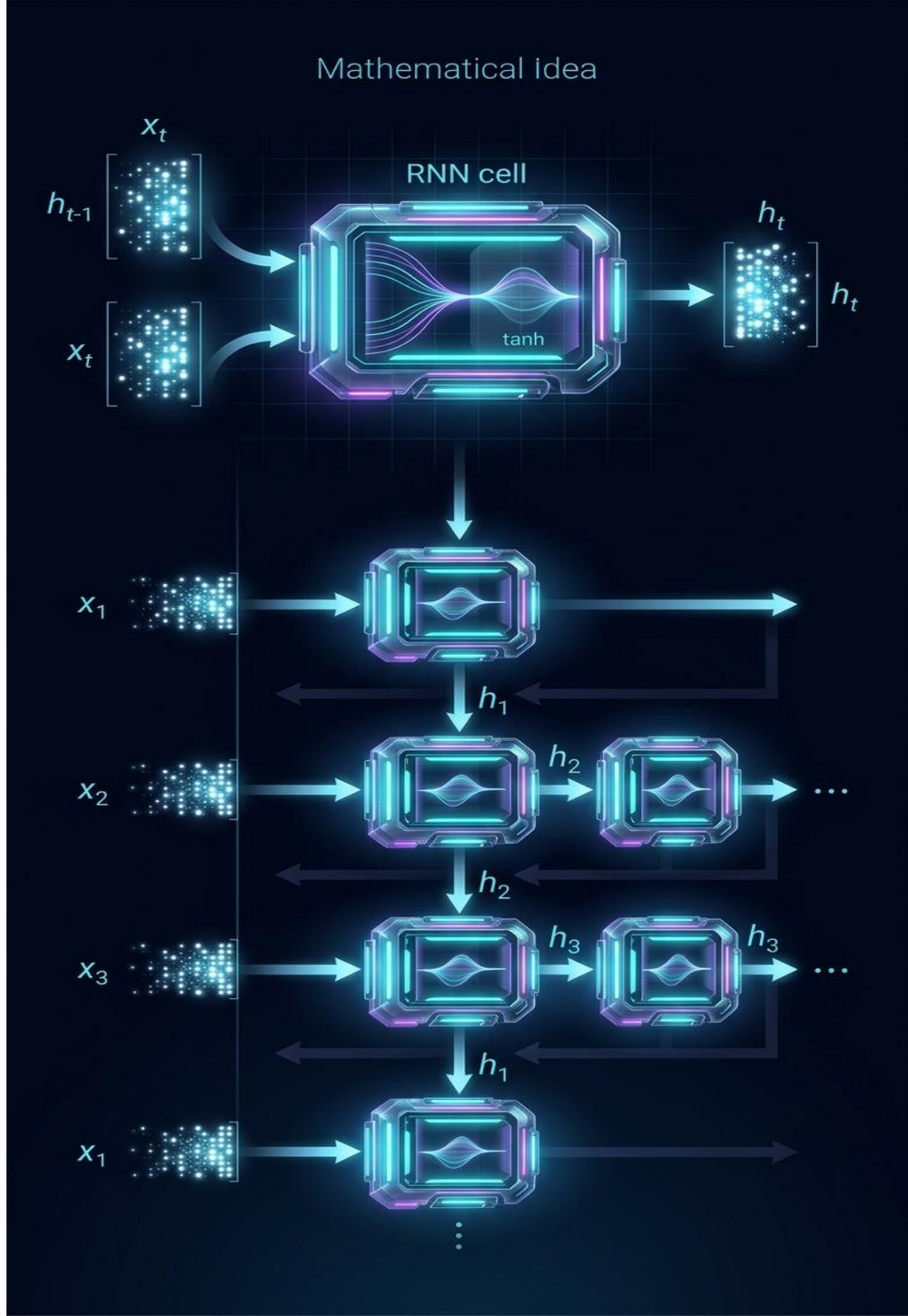


Basic RNN Computation

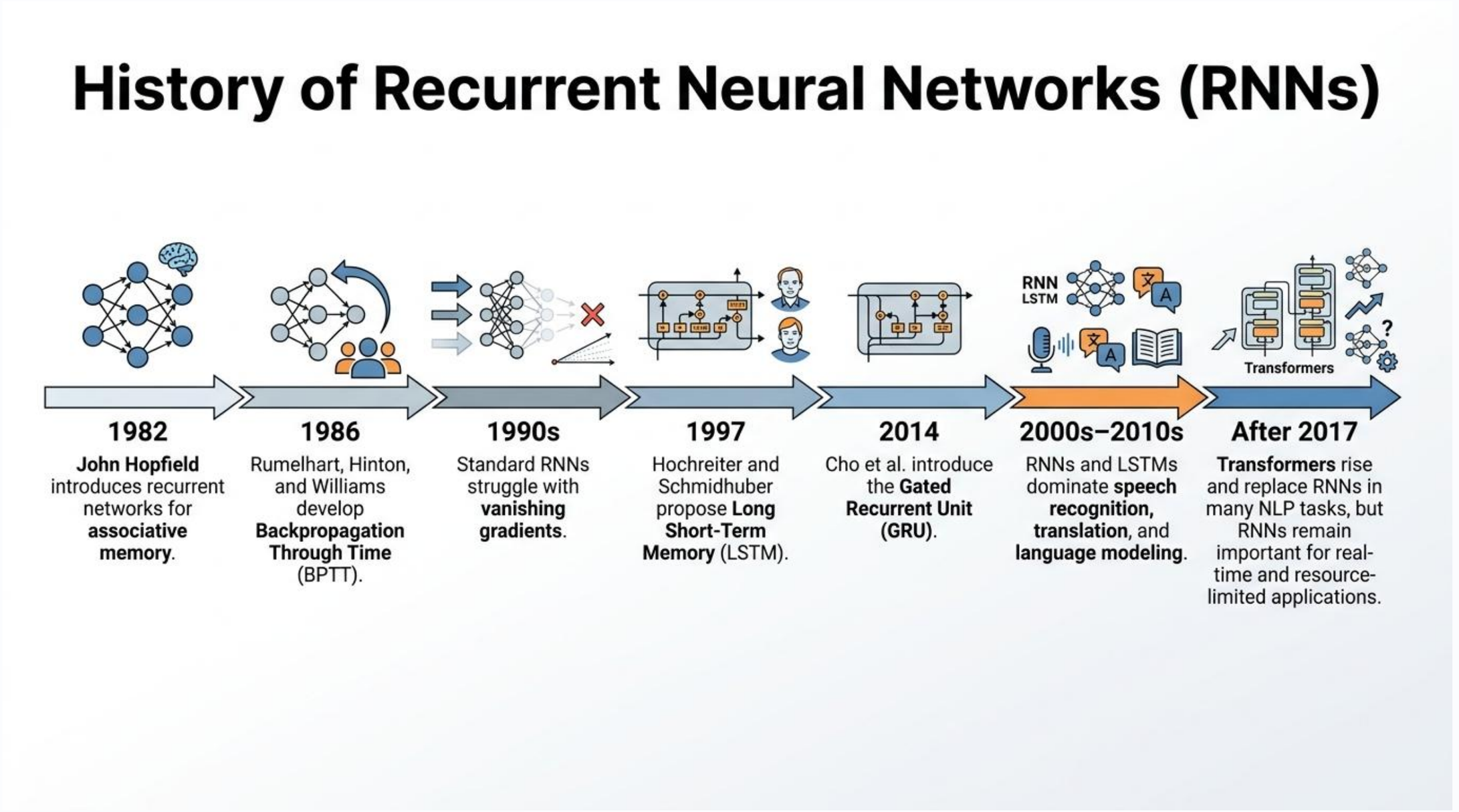
- At each time step t, the network receives an input vector x_t and updates its hidden state h_t using both the new input and the previous hidden state:
- $h_t = tan\,h(W_h h_{t-1} + W_x x_t + b)$ The hidden state acts as the network's internal memory. It contains information that influences future predictions. The output at each step can be computed from the hidden state, enabling the RNN to produce predictions at every point in the sequence.

Unrolled RNN Representation

- Although RNNs are recurrent, we can visualize them by "unrolling" the computation across the sequence. An unrolled RNN shows the same cell repeated at each time step, with each cell passing its hidden state to the next. This perspective helps us understand how information flows forward and how gradients flow backward during training.



RNN History


History of Recurrent Neural Networks (RNNs)
RNN
LSTM
Transformers
1982
John Hopfield introduces recurrent networks for associative memory.
1986
Rumelhart, Hinton, and Williams develop Backpropagation Through Time (BPTT).
1990s
Standard RNNs struggle with vanishing gradients.
1997
Hochreiter and Schmidhuber propose Long Short-Term Memory (LSTM).
2014
Cho et al. introduce the Gated Recurrent Unit (GRU).
2000s–2010s
RNNs and LSTMs dominate speech recognition, translation, and language modeling.
After 2017
Transformers rise and replace RNNs in many NLP tasks, but RNNs remain important for real-time and resource-limited applications.

CHAPTER 1

Recurrent Neural Networks

Challenges in learning of RNNs

MNNs with a sequenced input patterns

1. The sequenced input patterns may be combined with disturbance and non-related patterns.
2. The required features (to associate true output(outputs)) are hidden through sequenced inputs.
3. Among the sequenced input patterns, it may be short and long dependencies.
4. The sequenced input patterns may be presented with different modalities.

* RNNs make robust memories against distortion, disturbances and dimension variations of input patterns or different sampling rates.

Recurrent NNs

❑ Some known Recurrent NNs

- Recurrent Neural Networks(RNNs)
- Long-Short Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

❖ Such Networks make nonlinear non-homogenous difference Equations. By solving such equations, the nonlinear functions, which generate the output patterns based on the implicit sequenced input patterns, are revealed.

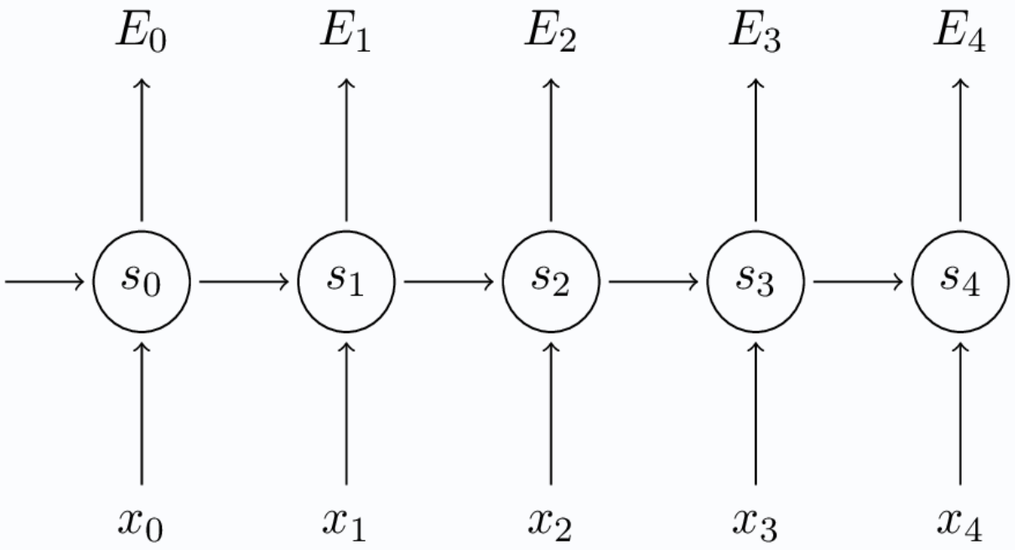
❖ Recurrent structures will make a plenty of memories utilized in pattern associations between implicit inputs and target outputs.

❑ Pattern Association (Memories) types.

- **One to one:** many classification problems: fingerprint/biometric signals/signature
- **Many to one:** voice/handwriting/…
- **One to Many-:** image captioning
- **Many to Many** -- Translation machines….text to picture/Film

Training RNNs

Back Propagation Through Time (BPTT)



$$s_t = \tanh(Ux_t + Ws_{t-1})$$
$$\hat{y}_t = \mathrm{softmax}(Vs_t)$$

Loss function

$$E_t(y_t, \hat{y}_t) = \underbrace{0.5(y_t - \hat{y}_t)^2}_{Squared\ Error}$$

or

$$E_t(y_t, \hat{y}_t) = \underbrace{-y_t \log(\hat{y}_t)}_{Cross\ entropy}$$

$$E(y, \hat{y}) = \sum_t E_t(y_t, \hat{y}_t)$$

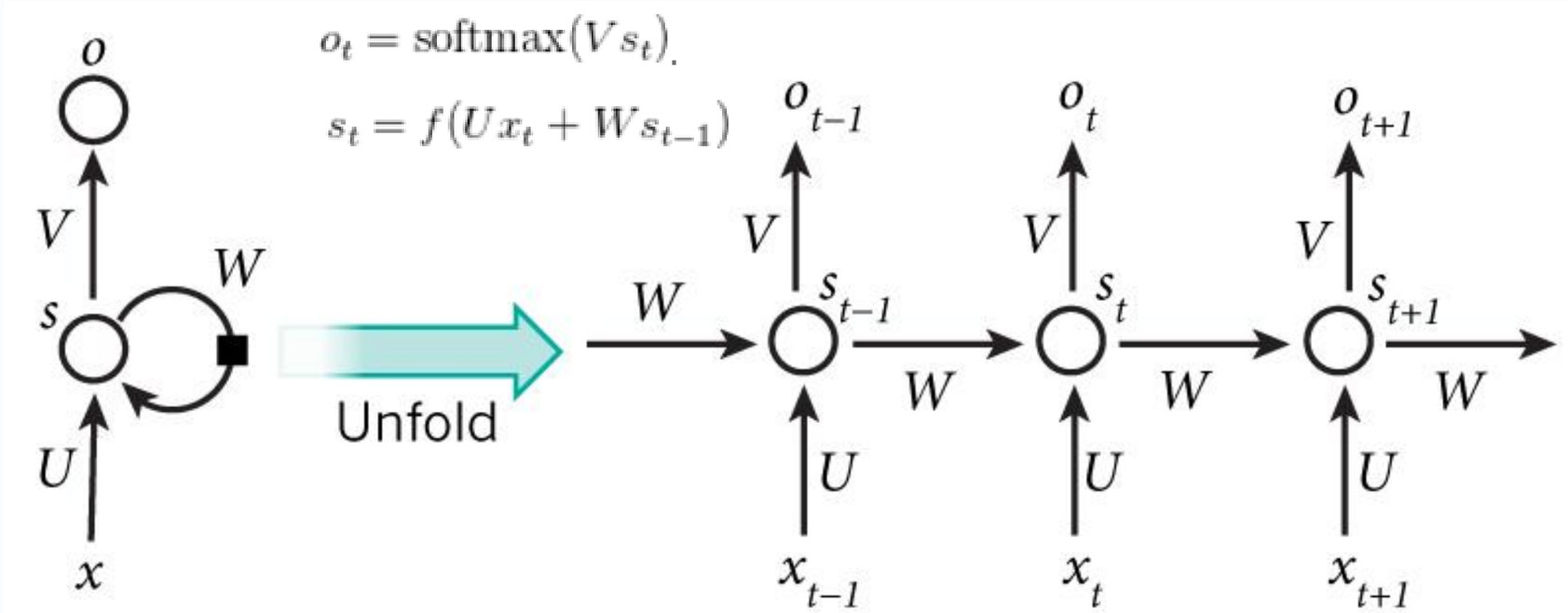
Updating Rules

$$W^+ = W^- - \eta \frac{\partial E_3}{\partial W}\Big|_{U^-, V^-, W^-}$$

$$U^+ = U^- - \eta \frac{\partial E_3}{\partial U}\Big|_{U^-, V^-, W^-}$$

$$V^+ = V^- - \eta \frac{\partial E_3}{\partial V}\Big|_{U^-, V^-, W^-}$$

A Typical simple RNN



- x_t is the input at time step t. For example, x_1 could be a one-hot vector corresponding to the second word of a sentence.
- s_t is the hidden state at time step t. It's the "memory" of the network. s_t is calculated based on the previous hidden state and the input at the current step: $s_t = f(Ux_t + Ws_{t-1})$. The function f usually is a nonlinearity such as tanh or ReLU. s_{-1}, which is required to calculate the first hidden state, is typically initialized to all zeroes.
- o_t is the output at step t. For example, if we wanted to predict the next word in a sentence it would be a vector of probabilities across our vocabulary. $o_t = \text{softmax}(Vs_t)$.

Back Propagation Through Time (BPTT)

$$s_t = \tanh(Ux_t + Ws_{t-1})$$
$$\hat{y}_t = \mathrm{softmax}(Vs_t)$$

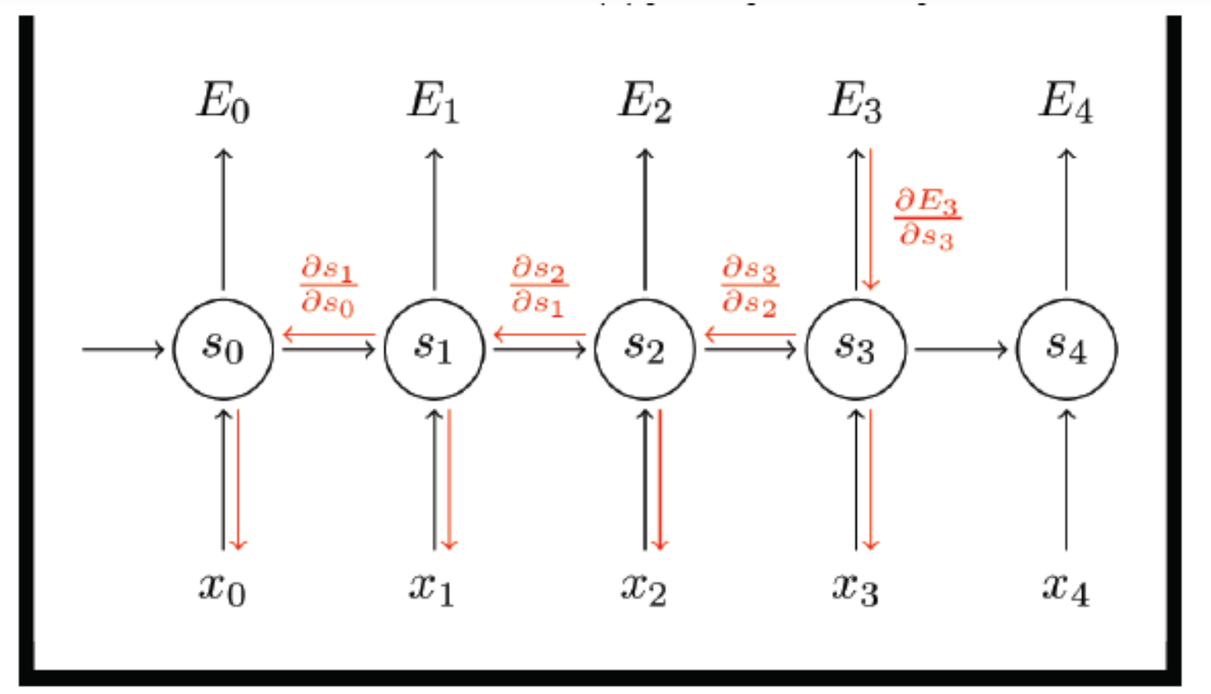
$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial s_3}\frac{\partial s_3}{\partial W}$$

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^{3}\frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial s_3}\frac{\partial s_3}{\partial s_k}\frac{\partial s_k}{\partial W}$$

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial s_3}\frac{\partial s_3}{\partial U}$$

$$\frac{\partial E_3}{\partial U} = \sum_{k=0}^{3}\frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial s_3}\frac{\partial s_3}{\partial s_k}\frac{\partial s_k}{\partial U}$$

$$\frac{\partial E_3}{\partial V} = \frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial V} = \frac{\partial E_3}{\partial \hat{y}_3}\frac{\partial \hat{y}_3}{\partial z_3}\frac{\partial z_3}{\partial V} = (\hat{y}_3 - y_3) \otimes s_3$$



Main limitation in RNN

derivative of sigmoid functions is less than one.

$$\frac{\partial \tanh(x)}{\partial x} < 1 \quad , \frac{\partial \mathrm{sigm}(x)}{\partial x} < 1$$

In “BPTT” when the RNN is unfolded for many times the back-propagated gradient coefficient is vanished for the inputs taken at older times.

$$\text{In computing } \left(\frac{\partial E_t}{\partial W}\right) \text{ or } \left(\frac{\partial E_t}{\partial U}\right) \text{ for } d \gg 1:$$

$$\left\| \frac{\partial E_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial s_t} \frac{\partial s_t}{\partial s_{t-1}} \cdots \frac{\partial s_{t-d+1}}{\partial s_{t-d}} \right\| \to 0$$

The learning for long dependencies is not effective any more.

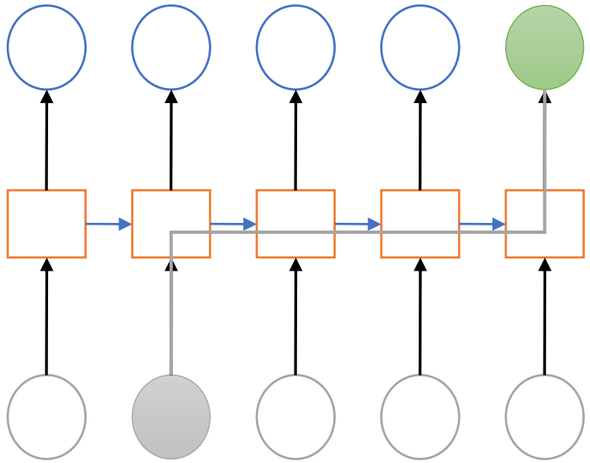
The Vanishing Gradient Problem

Error gradients vanish exponentially quickly with the size of the time lag between important events

RNNs are good to make Short Term Memory

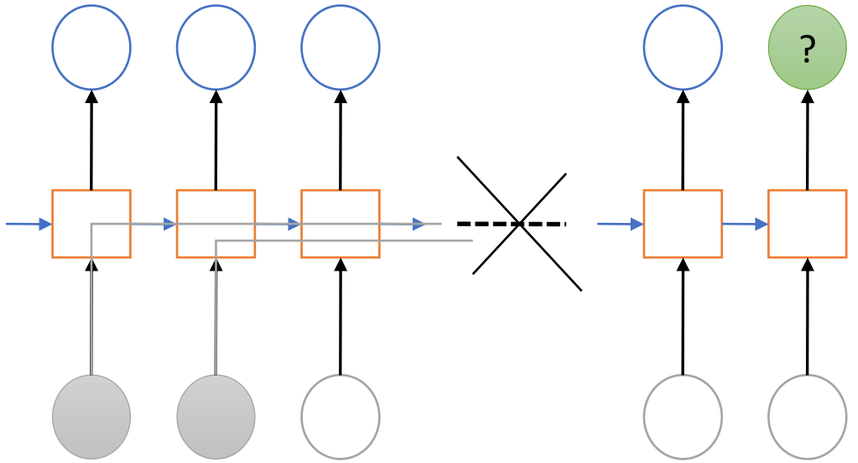
The clouds are in the SKY

implicit input → target



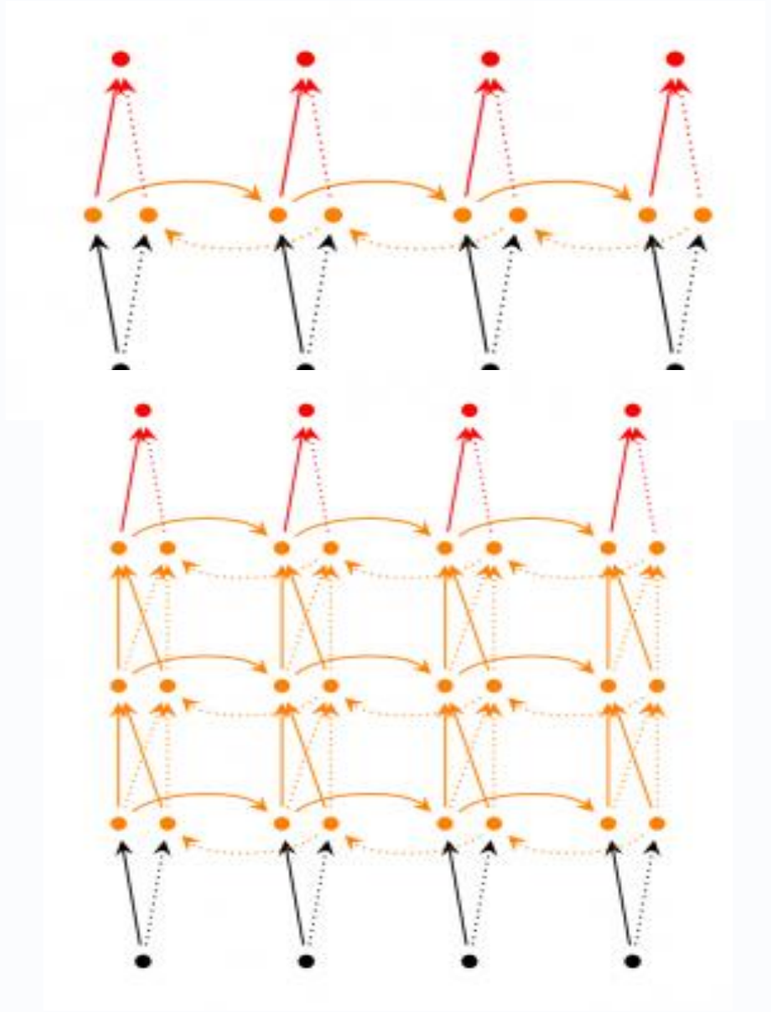
RNNs are not good to make Long Term Memory

I grew up in France……..I speak fluent French



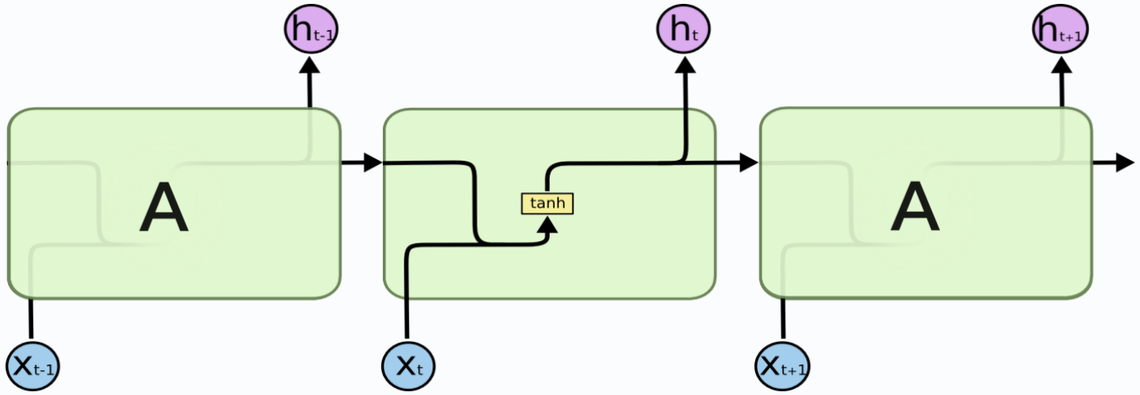
RNN Extensions

- **Bidirectional RNNs** are based on the idea that the output at time may not only depend on the previous elements in the sequence, but also future elements. For example, to predict a missing word in a sequence you want to look at both the left and the right context. Bidirectional RNNs are quite simple. They are just two RNNs stacked on top of each other. The output is then computed based on the hidden state of both RNNs.
- **Deep (Bidirectional) RNNs** are similar to Bidirectional RNNs, only that we now have multiple layers per time step. In practice this gives us a higher learning capacity (but we also need a lot of training data)



LSTM (long short-term memory)
Hochreitor & Shmidhuber 1997

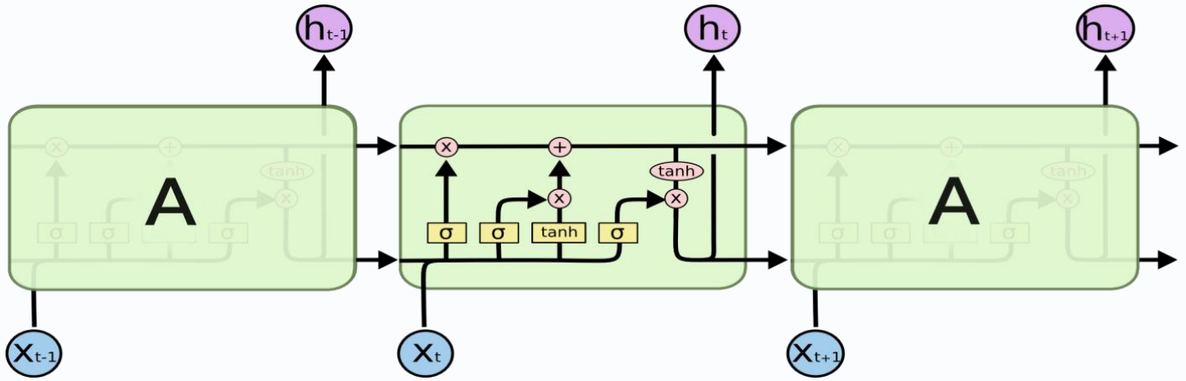
Module in RNN



($\mathbf{h_t}$ is the same $\mathbf{s_t}$)
$\mathbf{h_t} = \tanh(Ux_t + W\,\mathbf{h_{t-1}})$

The repeating module in a standard RNN contains a single layer.

Module in LSTM

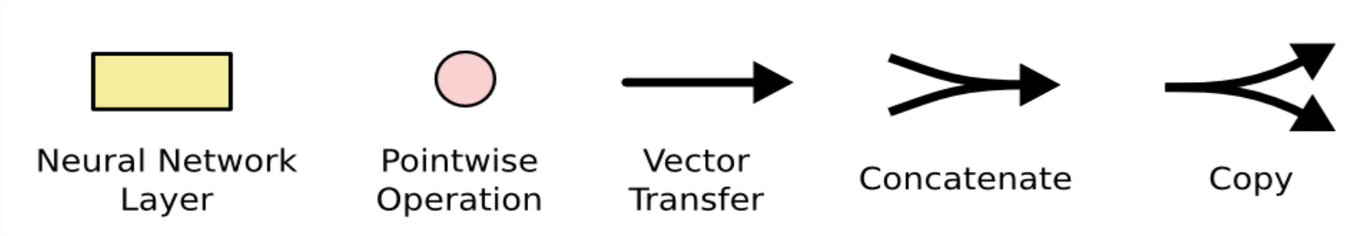


The repeating module in an LSTM contains four interacting layers.

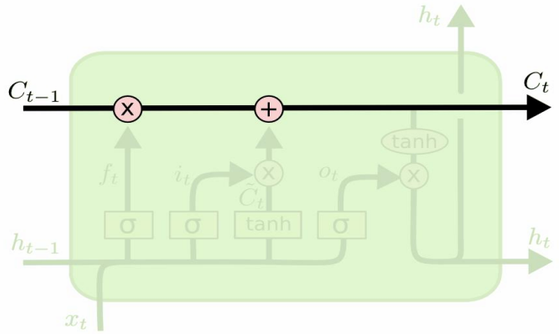
the LSTM can read, write and delete information from its memory.

Some Concepts

The notation

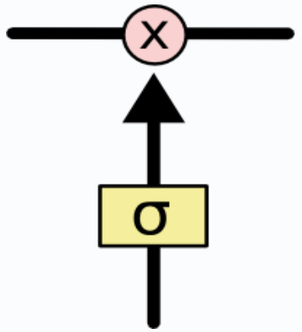


Two key concepts in LSTMs



1. Cell state

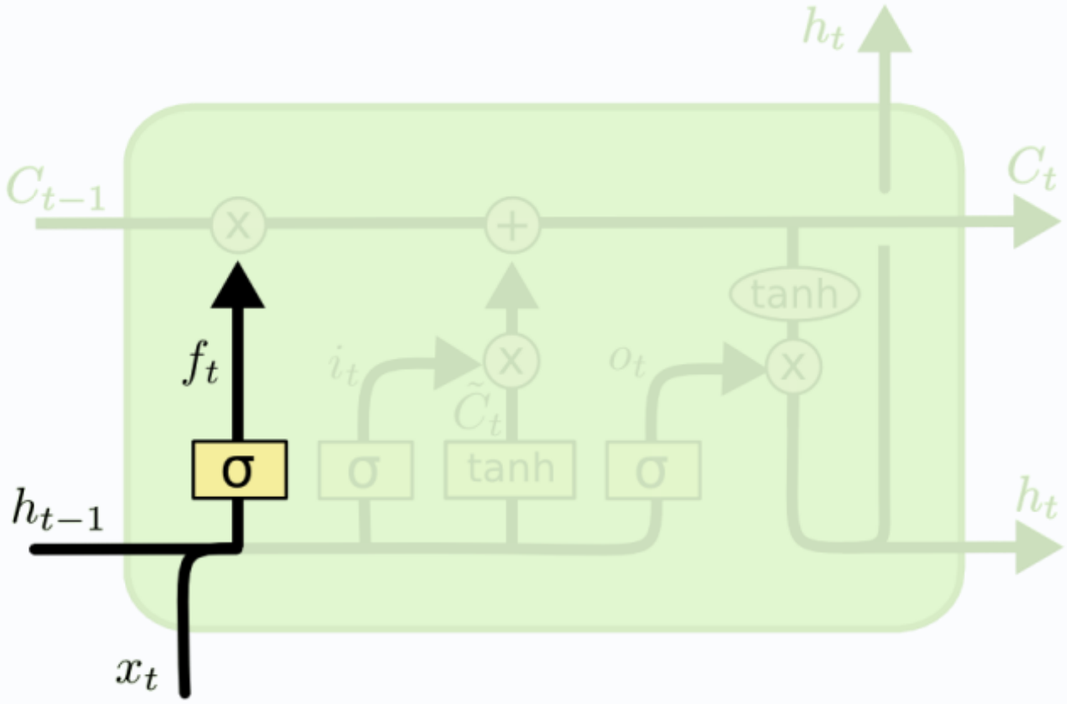
2. Gate



Gates are a way to optionally let information through.
They are composed out of a sigmoid neural net layer and a pointwise multiplication operation.

Step-by-Step LSTM Walk Through

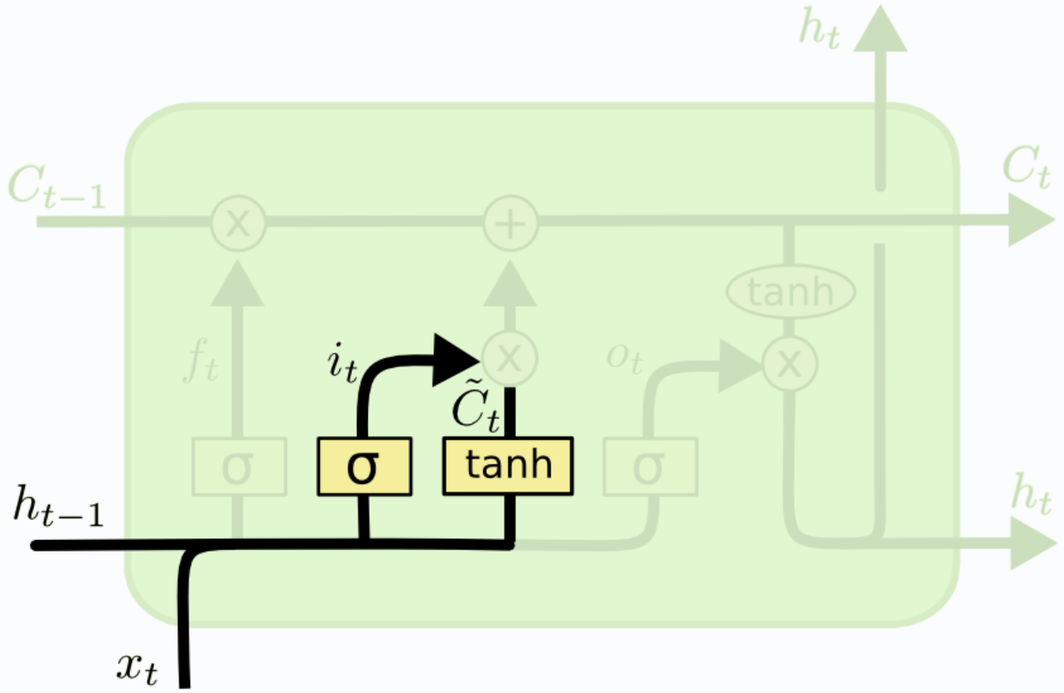
Step1: to decide what information we're going to throw away from the cell state.



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Step-by-Step LSTM Walk Through

Step2: to decide what new information we're going to store in the cell state.

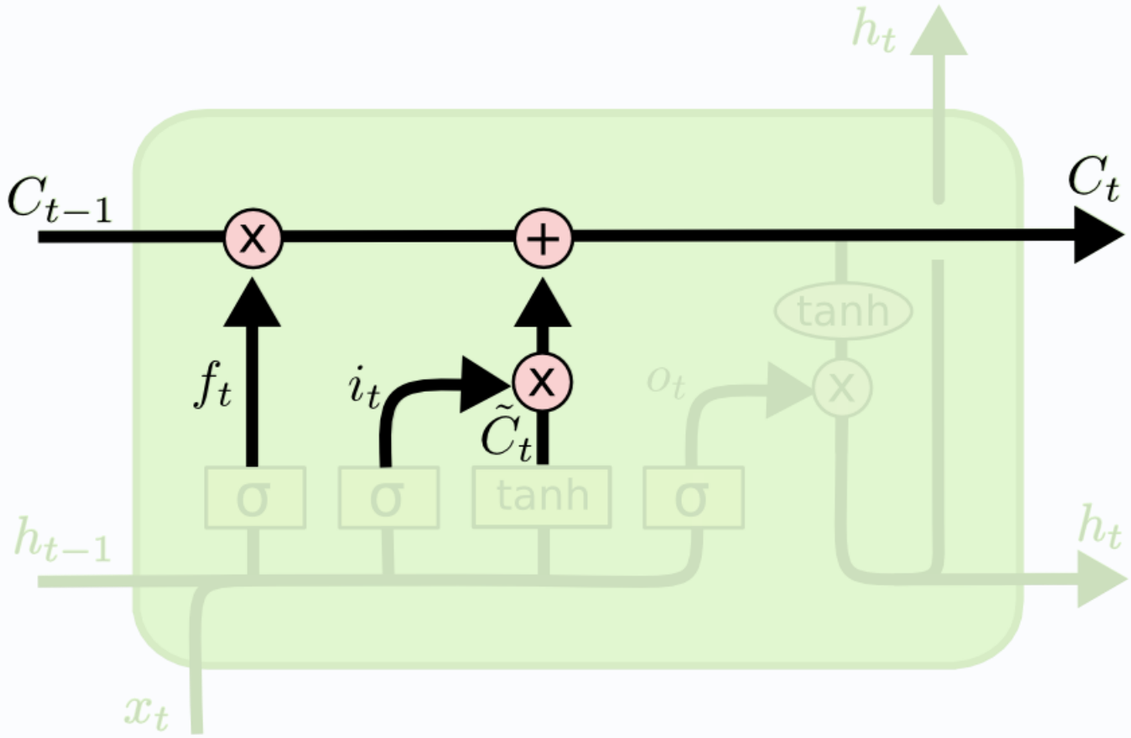


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Step-by-Step LSTM Walk Through

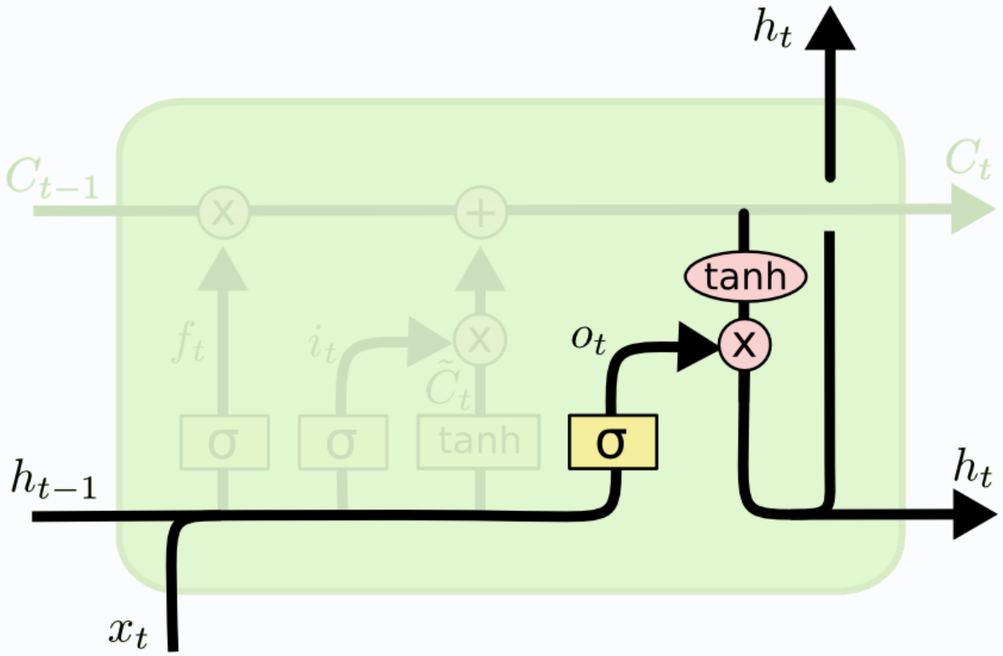
Step3: drop the information about the old subject's gender and add the new information, as we decided in the previous steps.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Step-by-Step LSTM Walk Through

Step4: to decide what we're going to output



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Compact Form Equations

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

$$\hat{y}_t = f(V.h_t + b_v)$$

$\boldsymbol{x_t \in \mathbf{R}^n}$

$\boldsymbol{W_{f,i,c,o} \in \mathbf{R}^{h \times (h+n)}}$

$\boldsymbol{b_{f,i,c,o} \in \mathbf{R}^{h \times 1}}$

$\boldsymbol{f_t, i_t, o_t, h_t. c_t \in \mathbf{R}^h}$

$\boldsymbol{V \in \mathbf{R}^{m \times h}} \quad \boldsymbol{\hat{y}_t, y_t \in \mathbf{R}^m}$

Parameters: $\{(W_i, b_i), (W_f, b_f), (W_C, b_c), (W_o, b_o), (V, b_V)\}$

LSTM

$$\text{Loss Function: } E_t = \sum_t E(y_t - \hat{y}_t)$$

$$\text{Parameters}^{+} = \text{Parameters}^{-} - \gamma \frac{\partial \text{E}_\text{t}}{\partial \text{Parameters}}$$

- Parameters are updated by “BPTT”

All computed gradients of parameters are added together through the time

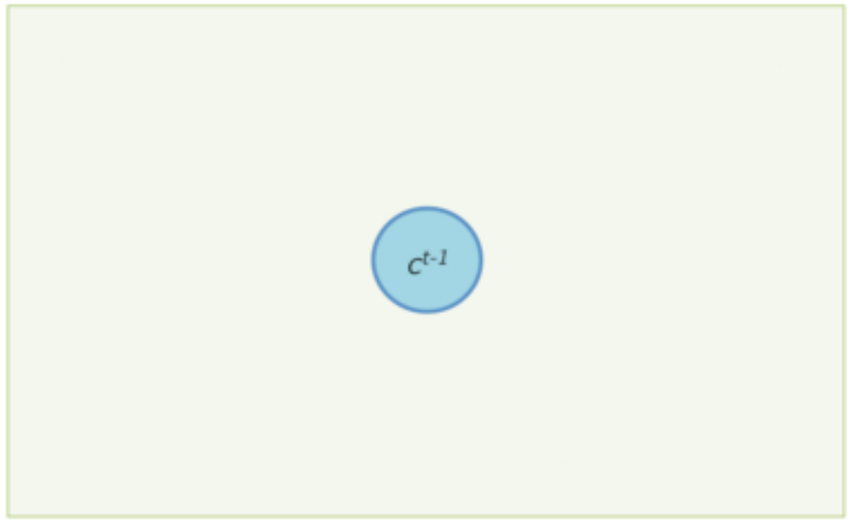
LSTM Learning methods

1. **Like RNN, "BPTT" can be performed to learn the weights and biases of a LSTM module. Unlike standard RNNs, the error remains in the unit's memory.**
2. LSTM can also be trained by a combination of artificial evolution for weights to the hidden units, and pseudo-inverse or support vector machines for weights to the output units.
3. In reinforcement learning applications LSTM can be trained by policy gradient methods,evolution strategies or genetic algorithms.

LSTM Forward and Backward Pass, Arun Mallya

Forward Pass: Initial State

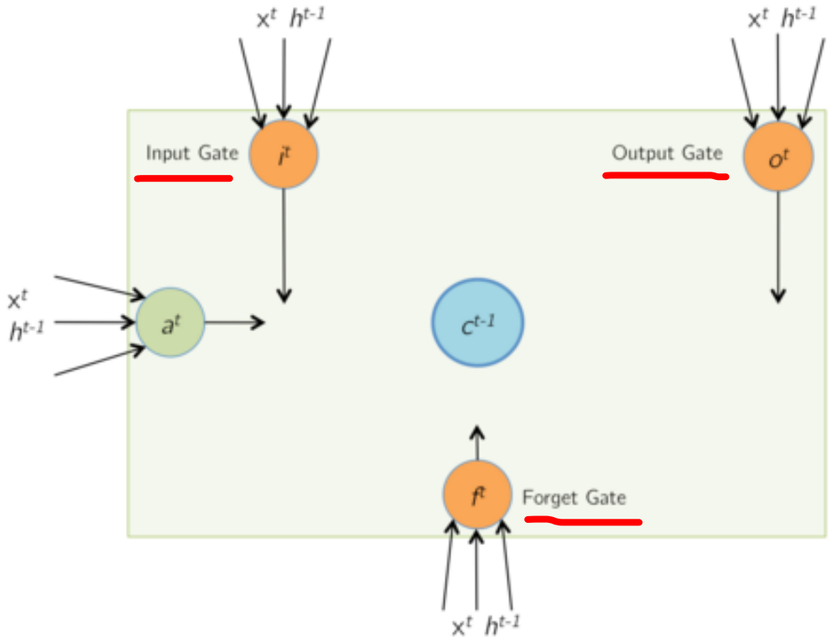
Initially, at time t, the memory cells of the LSTM contain values from the previous iteration at time (t − 1).



LSTM

Forward Pass: Input and Gate Computation

At time t, The LSTM receives a new input vector x^t (including the bias term), as well as a vector of its output at the previous timestep, h^{t-1}.



$$
\begin{aligned}
a^t &= \tanh(W_c x^t + U_c h^{t-1}) = \tanh(\hat{a}^t) \\
i^t &= \sigma(W_i x^t + U_i h^{t-1}) = \sigma(\hat{i}^t) \\
f^t &= \sigma(W_f x^t + U_f h^{t-1}) = \sigma(\hat{f}^t) \\
o^t &= \sigma(W_o x^t + U_o h^{t-1}) = \sigma(\hat{o}^t)
\end{aligned}
$$

Ignoring the non-linearities,

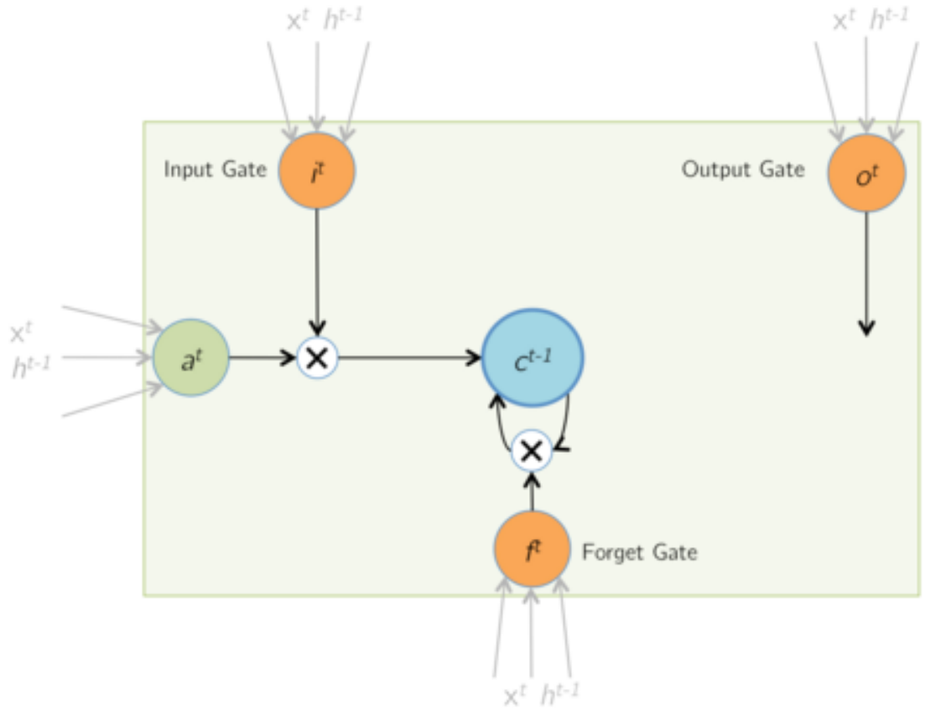
$$
z^t = \begin{bmatrix} \hat{a}^t \\ \hat{i}^t \\ \hat{f}^t \\ \hat{o}^t \end{bmatrix} = \begin{bmatrix} W^c & U^c \\ W^i & U^i \\ W^f & U^f \\ W^o & U^o \end{bmatrix} \times \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix} = W \times I^t
$$

If the input x^t is of size $n \times 1$, and we have d memory cells, then the size of each of W_* and U_* is $d \times n$, and and $d \times d$ resp. The size of W will then be $4d \times (n + d)$. Note that each one of the d memory cells has its own weights W_* and U_*, and that the only time memory cell values are shared with other LSTM units is during the product with U_*.

LSTM

Forward Pass: Memory Cell Update

During this step, the values of the memory cells are updated with a combination of a^t, and the previous cell contents c^{t-1}. The combination is based on the magnitudes of the input gate i^t and the forget gate f^t. $\odot$ denotes elementwise product (Hadamard product).

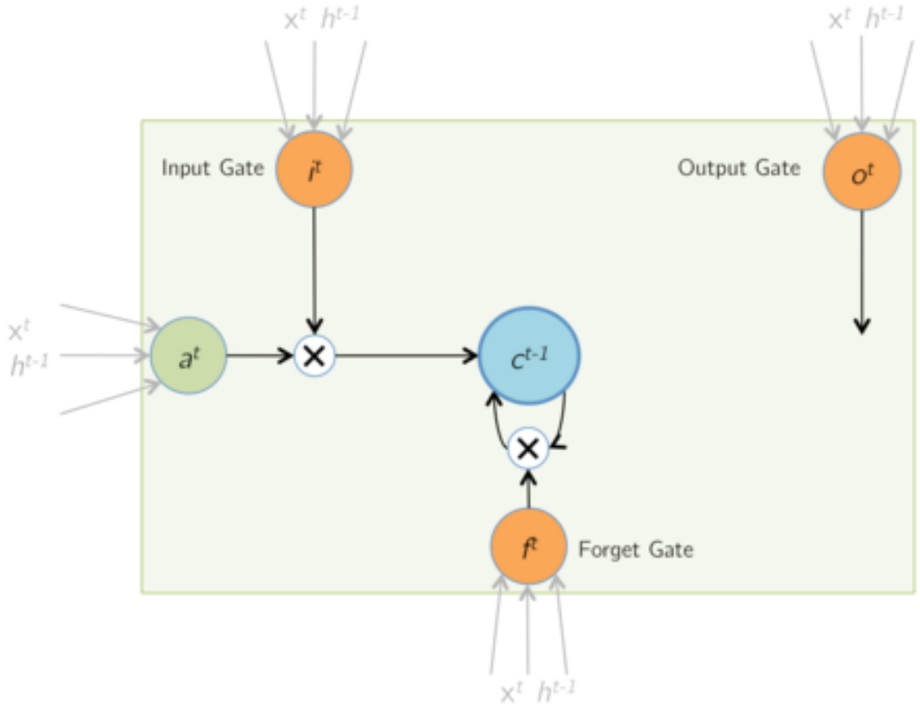


$$c^t = i^t \odot a^t + f^t \odot c^{t-1}$$

LSTM

Forward Pass: Memory Cell Update

During this step, the values of the memory cells are updated with a combination of a^t, and the previous cell contents c^{t-1}. The combination is based on the magnitudes of the input gate i^t and the forget gate f^t. $\odot$ denotes elementwise product (Hadamard product).

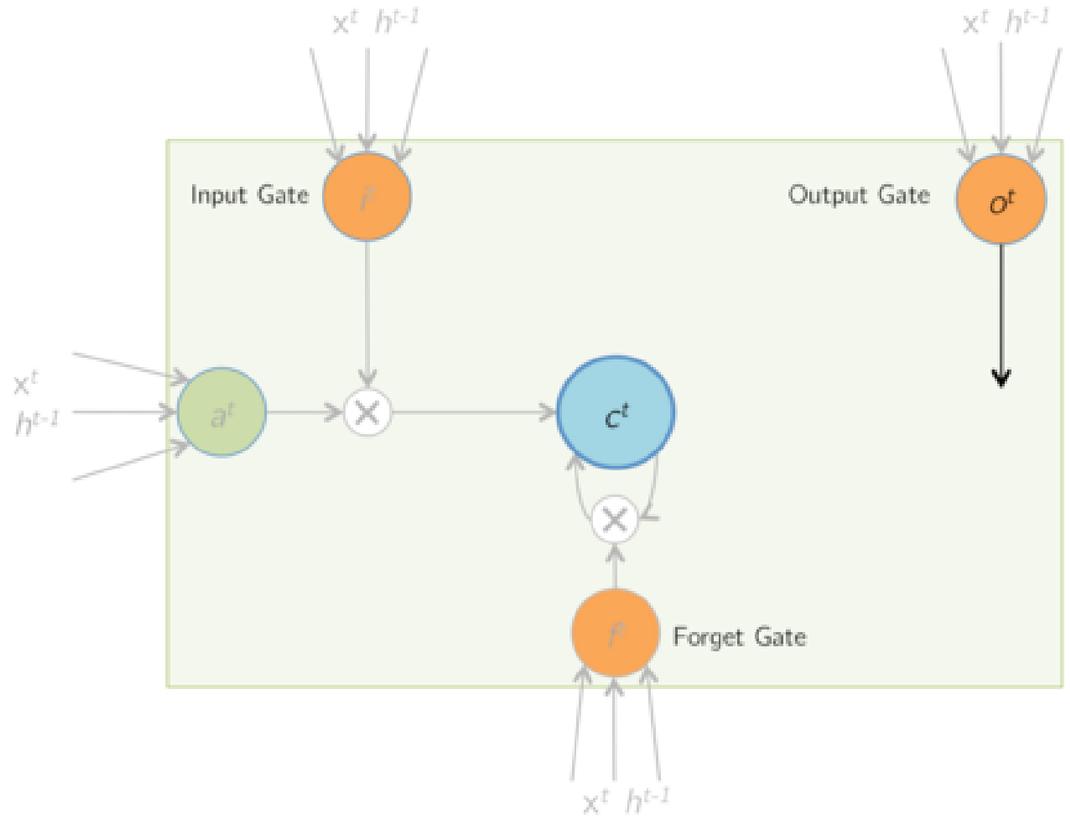


$$c^t = i^t \odot a^t + f^t \odot c^{t-1}$$

LSTM

Forward Pass: Updated Memory Cells

The contents of the memory cells are updated to the latest values.

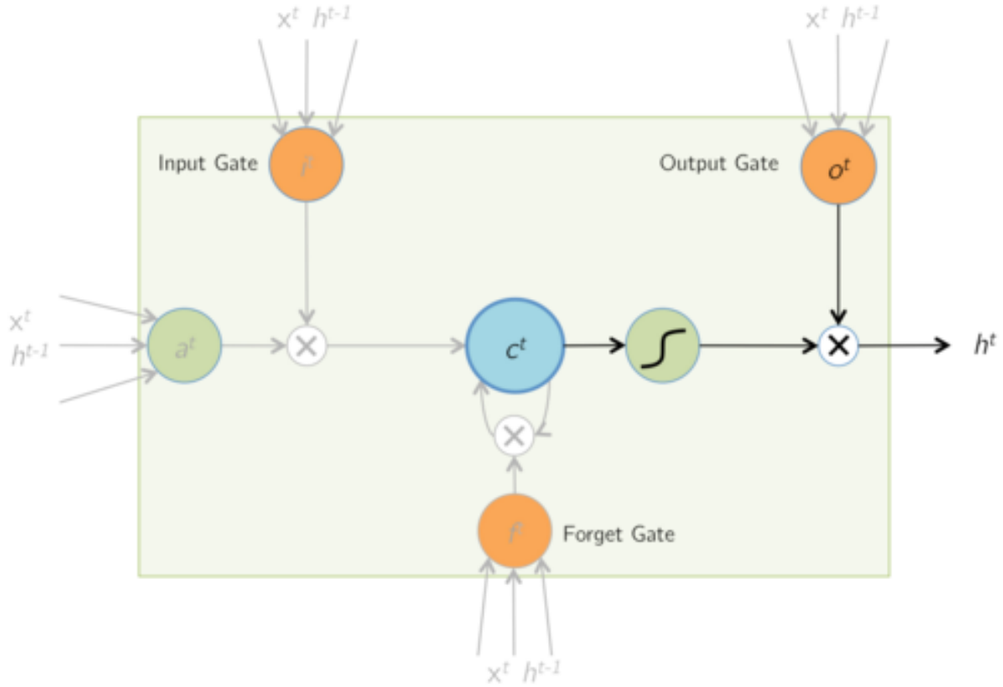


$$c^{t-1} \rightarrow c^t$$

LSTM

Forward Pass: Output

Finally, the LSTM cell computes an output value by passing the updated (and current) cell value through a non-linearity. The output gate determines how much of this computed output is actually passed out of the cell as the final output h^t.

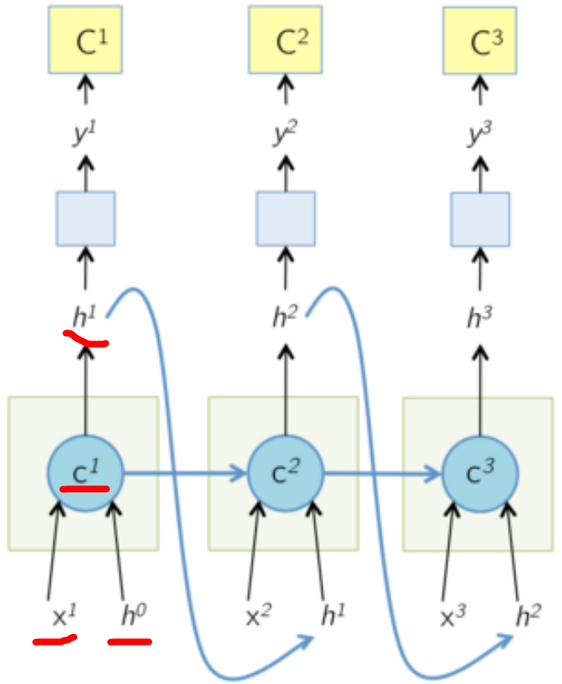


$$h^t = o^t \odot \tanh(c^t)$$

LSTM

Forward Pass: Unrolled Network

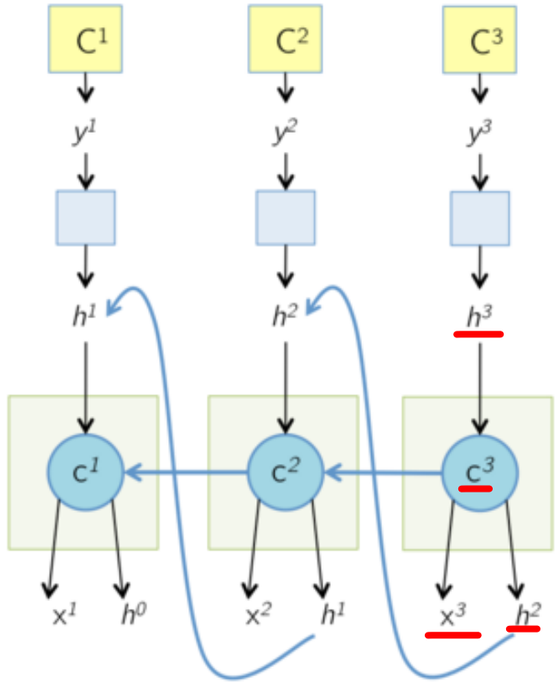
The unrolled network during the forward pass is shown below. Note that the gates have not been shown for brevity. An interesting point to note here is that in the computational graph below, the cell state at time T, c^T is responsible for computing h^T as well as the next cell state c^{T+1}. At each time step, the cell output h^T is shown to be passed to some more layers on which a cost function C^T is computed, as the way an LSTM would be used in a typical application like captioning or language modeling.



LSTM

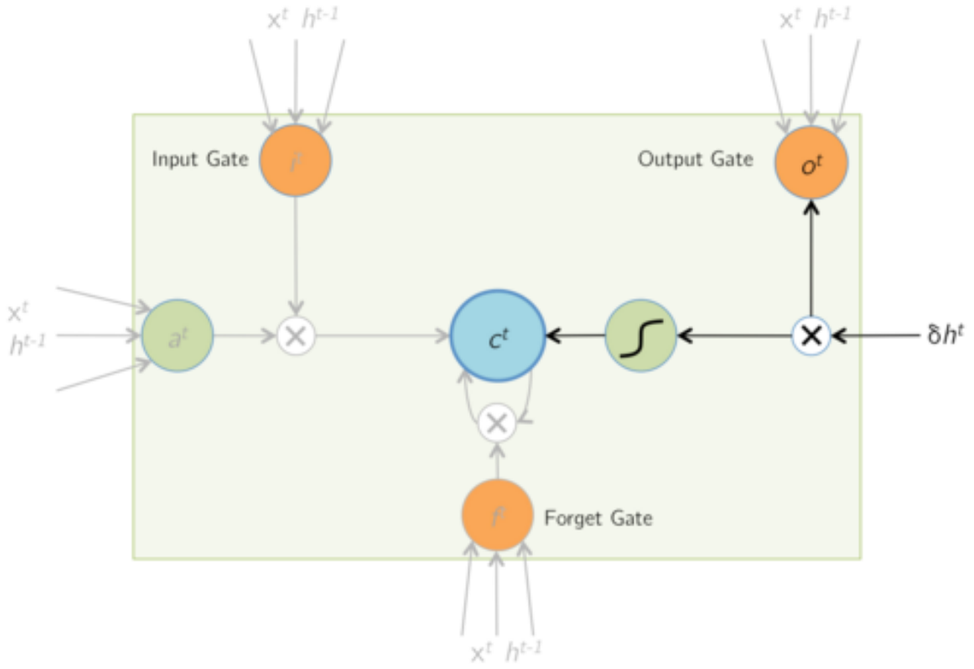
Backward Pass: Unrolled Network

The unrolled network during the backward pass is shown below. All the arrows in the previous slide have now changed their direction. The cell state at time T, c^T receives gradients from h^T as well as the next cell state c^{T+1}. The next few slides focus on computing these two gradients. At any time step T, these two gradients are accumulated before being backpropagated to the layers below the cell and the previous time steps.



LSTM

Backward Pass: Output



Forward Pass: $h^t = o^t \odot \tanh(c^t)$

Given $\delta h^t = \frac{\partial E}{\partial h^t}$, find $\delta o^t, \delta c^t$

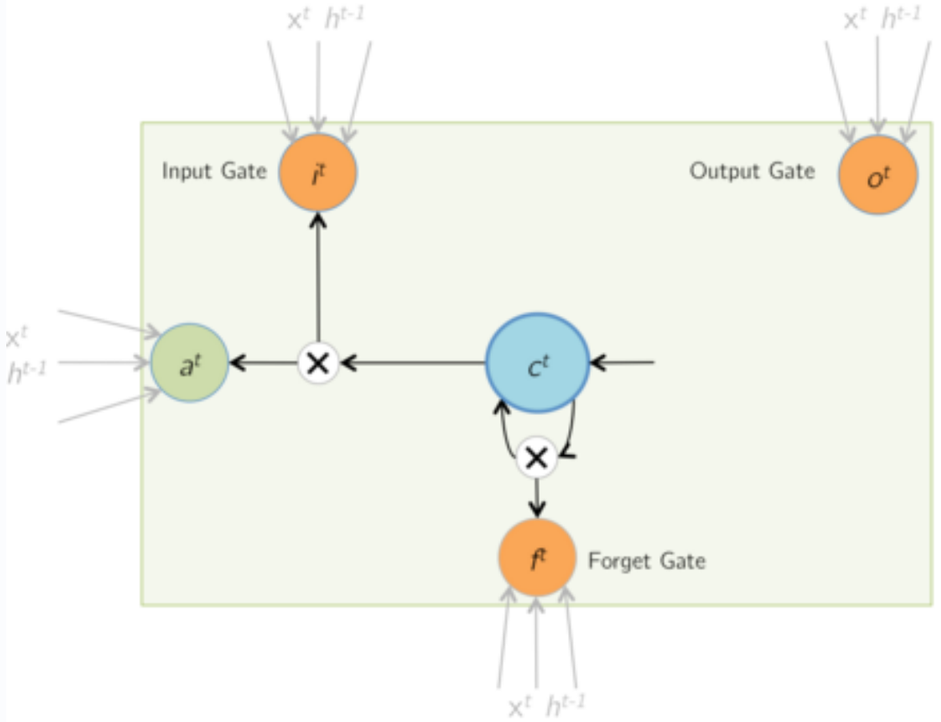
$$\frac{\partial E}{\partial o_i^t} = \frac{\partial E}{\partial h_i^t} \cdot \frac{\partial h_i^t}{\partial o_i^t}$$
$$= \delta h_i^t \cdot \tanh(c_i^t)$$
$$\therefore \delta o^t = \delta h^t \odot \tanh(c^t)$$

$$\frac{\partial E}{\partial c_i^t} = \frac{\partial E}{\partial h_i^t} \cdot \frac{\partial h_i^t}{\partial c_i^t}$$
$$= \delta h_i^t \cdot o_i^t \cdot (1 - \tanh^2(c_i^t))$$
$$\therefore \delta c^t \mathrel{+}= \delta h^t \odot o^t \odot (1 - \tanh^2(c^t))$$

Note that the $+=$ above is so that this gradient is added to gradient from time step $(t+1)$ (calculated on next slide, refer to the gradient accumulation mentioned in the previous slide)

LSTM

Backward Pass: LSTM Memory Cell Update



Forward Pass: $c^t = i^t \odot a^t + f^t \odot c^{t-1}$

Given $\delta c^t = \frac{\partial E}{\partial c^t}$, find $\delta i^t, \delta a^t, \delta f^t, \delta c^{t-1}$

$$\frac{\partial E}{\partial i_i^t} = \frac{\partial E}{\partial c_i^t} \cdot \frac{\partial c_i^t}{\partial i_i^t}$$
$$= \delta c_i^t \cdot a_i^t$$
$$\therefore \delta i^t = \delta c^t \odot a^t$$

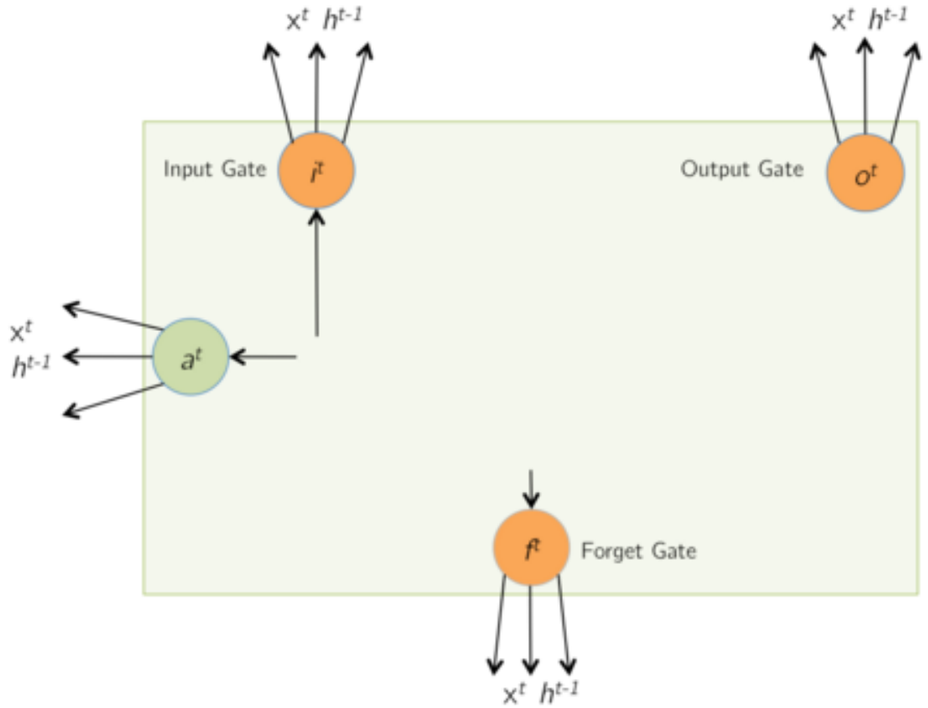
$$\frac{\partial E}{\partial f_i^t} = \frac{\partial E}{\partial c_i^t} \cdot \frac{\partial c_i^t}{\partial f_i^t}$$
$$= \delta c_i^t \cdot c_i^{t-1}$$
$$\therefore \delta f^t = \delta c^t \odot c^{t-1}$$

$$\frac{\partial E}{\partial a_i^t} = \frac{\partial E}{\partial c_i^t} \cdot \frac{\partial c_i^t}{\partial a_i^t}$$
$$= \delta c_i^t \cdot i_i^t$$
$$\therefore \delta a^t = \delta c^t \odot i^t$$

$$\frac{\partial E}{\partial c_i^{t-1}} = \frac{\partial E}{\partial c_i^t} \cdot \frac{\partial c_i^t}{\partial c_i^{t-1}}$$
$$= \delta c_i^t \cdot f_i^t$$
$$\therefore \delta c^{t-1} = \delta c^t \odot f^t$$

LSTM

Backward Pass: Input and Gate Computation - II



Forward Pass: $z^t = W \times I^t$

Given δz^t, find $\delta W^t, \delta h^{t-1}$

$$\delta I^t = W^T \times \delta z^t$$

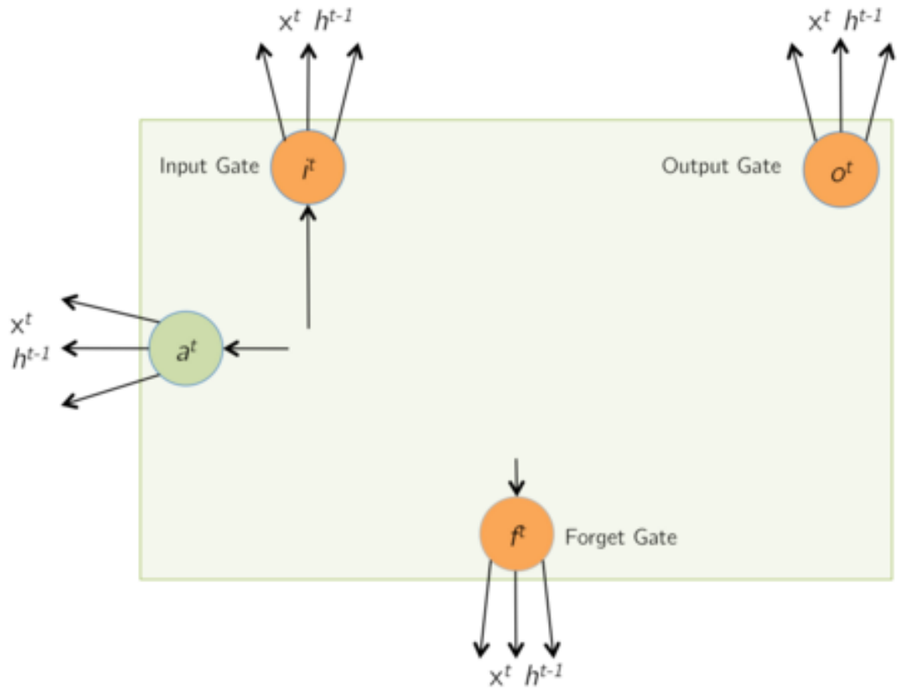
As $I^t = \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}$,

δh^{t-1} can be retrieved from δI^t

$$\delta W^t = \delta z^t \times (I^t)^T$$

LSTM

Backward Pass: Input and Gate Computation - II



Forward Pass: $z^t = W \times I^t$

Given δz^t, find $\delta W^t, \delta h^{t-1}$

$$\delta I^t = W^T \times \delta z^t$$

As $I^t = \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix}$,

δh^{t-1} can be retrieved from δI^t

$$\delta W^t = \delta z^t \times (I^t)^T$$

LSTM

Parameter Update

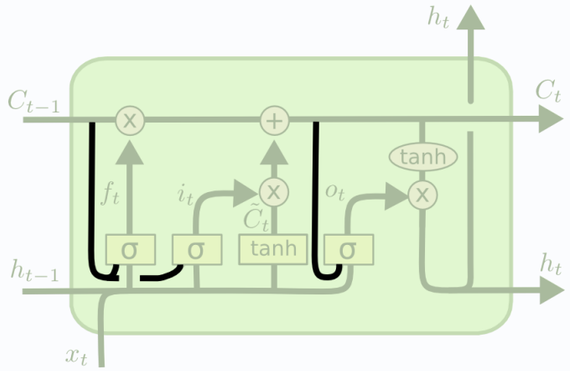
If input x has T time-steps, i.e. $x = [x^1, x^2, \cdots, x^T]$, then

$$\delta W = \sum_{t=1}^{T} \delta W^t$$

W is then updated using an appropriate Stochastic Gradient Descent solver.

Extended Versions

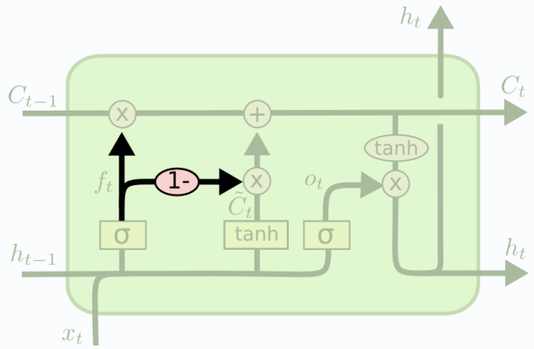
1. One popular LSTM variant, introduced by Gers & Schmidhuber (2000), is adding "peephole connections." This means that we let the gate layers look at the cell state.



$$f_t = \sigma\left(W_f \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_f\right)$$
$$i_t = \sigma\left(W_i \cdot [\boldsymbol{C_{t-1}}, h_{t-1}, x_t] \; + \; b_i\right)$$
$$o_t = \sigma\left(W_o \cdot [\boldsymbol{C_t}, h_{t-1}, x_t] \; + \; b_o\right)$$

Extended Versions

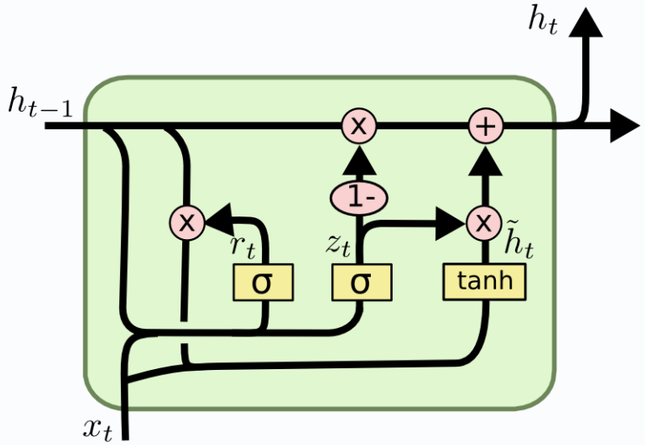
2. Another variation is to use coupled forget and input gates. Instead of separately deciding what to forget and what we should add new information to, we make those decisions together. We only forget when we're going to input something in its place. We only input new values to the state when we forget something older.



$$C_t = f_t * C_{t-1} + (1 - \boldsymbol{f_t}) * \tilde{C}_t$$

Extended Versions

2. A slightly more dramatic variation on the LSTM is the Gated Recurrent Unit, or GRU, introduced by Cho, et al. (2014). It combines the forget and input gates into a single "update gate." It also merges the cell state and hidden state, and makes some other changes. The resulting model is simpler than standard LSTM models, and has been growing increasingly popular.



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Types of Recurrent Neural Networks

1. Vanilla (Simple) RNN

The basic form of RNN where the hidden state at each time step is computed as a nonlinear function of the current input and the previous hidden state. It is easy to implement but suffers from vanishing and exploding gradients for long sequences.

2. One-to-One, One-to-Many, Many-to-One, Many-to-Many

RNNs can be configured in different input–output structures:

- *One-to-One*: A single input mapped to a single output (not really recurrent).
- *One-to-Many*: A single input leading to a sequence of outputs (e.g., image captioning).
- *Many-to-One*: A sequence of inputs leading to a single output (e.g., sentiment classification).
- *Many-to-Many*: A sequence of inputs mapped to a sequence of outputs (e.g., machine translation).

3. Bidirectional RNN (BiRNN)

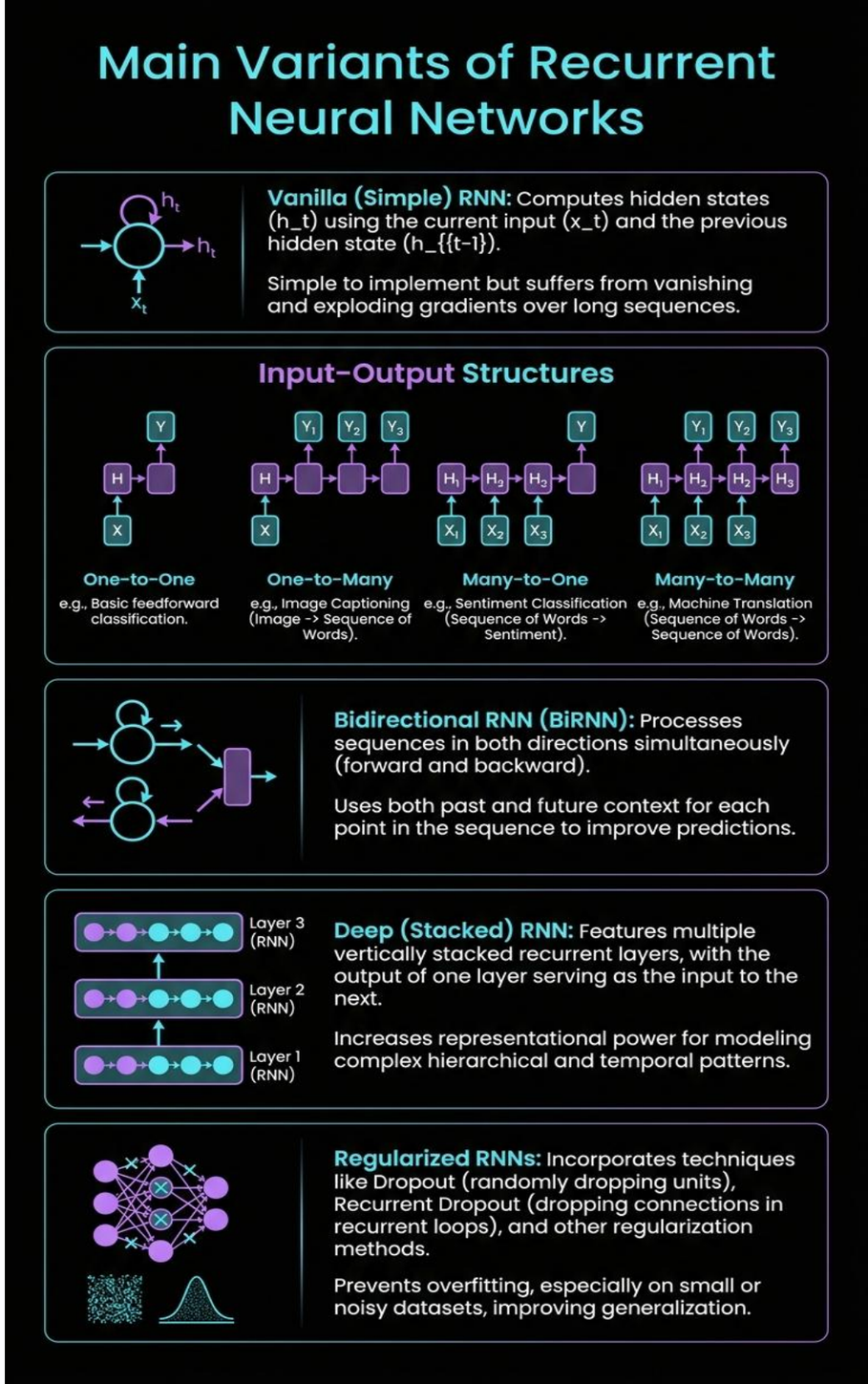
Uses two RNNs: one processes the sequence forward (from past to future), and the other backward (from future to past). Their hidden states are combined, allowing the model to use both past and future context for each time step. This is useful when the full sequence is available, such as in offline text processing.

4. Deep (Stacked) RNN

Multiple recurrent layers are stacked on top of each other. The hidden states from one layer become the inputs to the next layer. This increases the representational power and helps the network capture more complex temporal patterns, but also makes training more challenging.

5. Regularized RNNs

Variants of RNNs that include dropout, recurrent dropout, or other regularization techniques to prevent overfitting and improve generalization, particularly on small or noisy datasets.



Types of LSTM Networks

1. Vanilla (Standard) LSTM

The basic LSTM cell with three main gates: input gate, forget gate, and output gate, along with a cell state and hidden state. It effectively mitigates the vanishing gradient problem by enabling long-range information flow through the cell state.

2. Peephole LSTM

In peephole LSTMs, the gates (input, forget, and/or output) receive direct connections from the cell state. This allows the gates to see the internal memory when making decisions, potentially improving performance on tasks requiring precise timing or counting.

3. Coupled Input–Forget Gate LSTM

In this variant, the input gate and forget gate are coupled so that one gate's activation can be derived from the other (e.g., $f_t = 1 - i_t$). This reduces the number of parameters and simplifies the gating mechanism, while often maintaining similar performance.

4. Bidirectional LSTM (BiLSTM)

Combines two LSTM networks: one processes the sequence from left to right and the other from right to left. The outputs (or hidden states) from both directions are concatenated or combined. BiLSTMs are widely used in NLP tasks where both past and future context improve understanding.

5. Stacked (Deep) LSTM

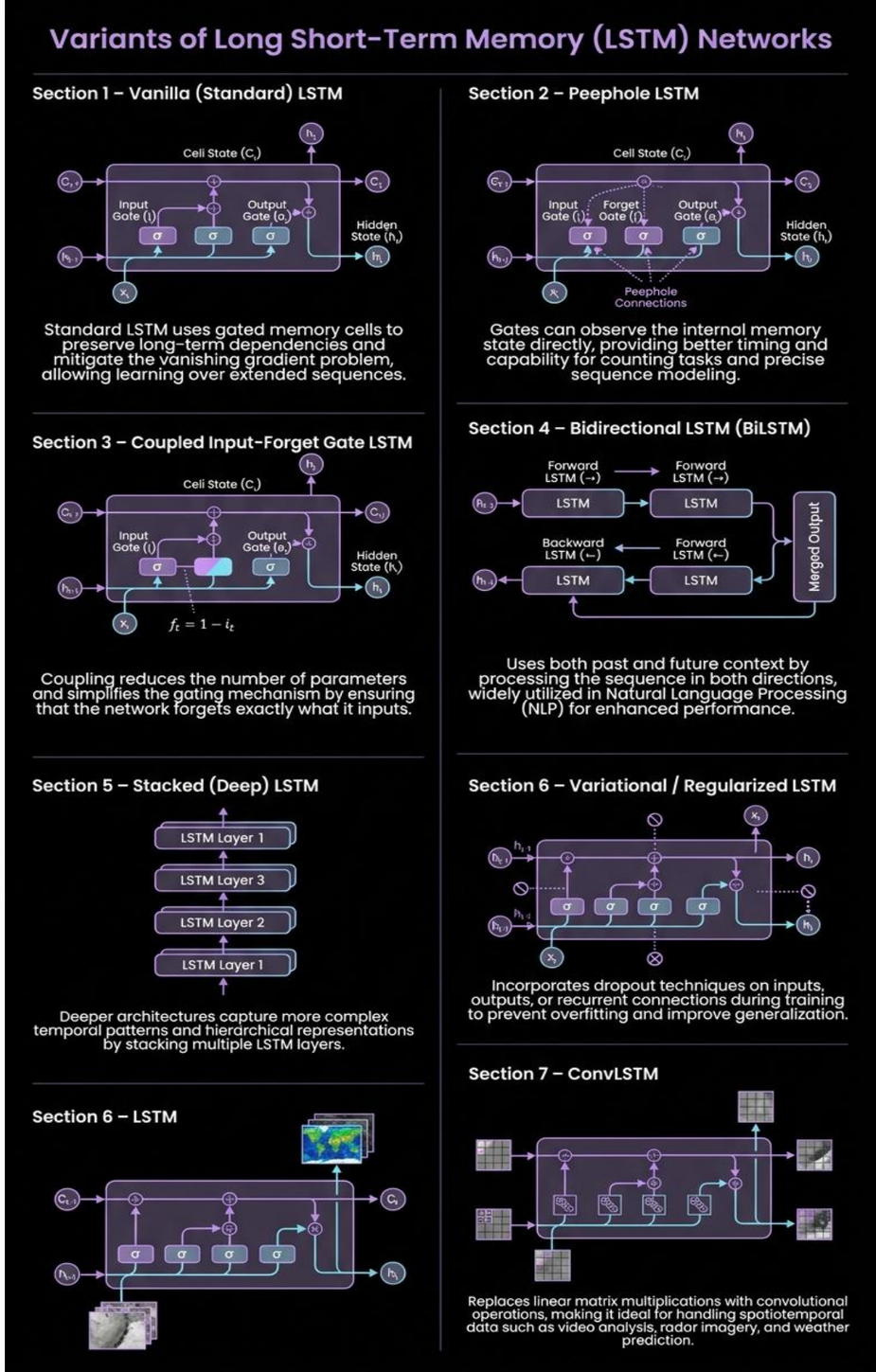
Multiple LSTM layers are stacked vertically. The hidden states from one layer serve as inputs to the next layer. Deep LSTMs can model highly complex temporal structures, but they require more data and careful regularization.

6. Variational / Regularized LSTM

LSTMs with dropout applied to inputs, outputs, or recurrent connections (e.g., variational dropout) to improve generalization. These are especially useful for large models and limited training data.

7. ConvLSTM (Convolutional LSTM)

A specialized LSTM where linear transformations are replaced with convolutions, typically used for spatiotemporal data such as video sequences, radar data, or weather prediction grids.



Types of Gated Recurrent Units

1. Vanilla (Standard) GRU

The basic GRU cell uses two gates: an update gate and a reset gate. It merges the cell state and hidden state into a single vector, making the architecture simpler and often faster to train than LSTMs, while achieving comparable performance on many tasks.

2. Bidirectional GRU (BiGRU)

Similar to Bidirectional LSTMs, BiGRUs use two GRU networks that process the sequence in opposite directions. The forward and backward hidden states are combined to leverage both past and future context. This configuration is widely used in text and speech applications.

3. Stacked (Deep) GRU

Multiple GRU layers are stacked on top of each other. The output of one GRU layer becomes the input to the next. Deep GRUs can capture hierarchical temporal features and are useful when the task requires modeling complex sequence dependencies.

4. Convolutional GRU (ConvGRU)

A variant where recurrent operations are implemented using convolutions instead of fully connected layers. ConvGRUs are designed for spatiotemporal sequence modeling, such as video processing, dynamic scenes, and physical system simulations on grids.

5. Regularized GRU

GRUs with dropout on input or recurrent connections, weight decay, or other regularization techniques to combat overfitting. These are important when training deep or wide GRU networks on limited data.

6. Minimal / Light-weight GRU Variants

Some research introduces simplified GRU cells that further reduce parameters or modify gating equations for efficiency on edge devices or real-time applications. These trade slight performance for better speed and memory usage.